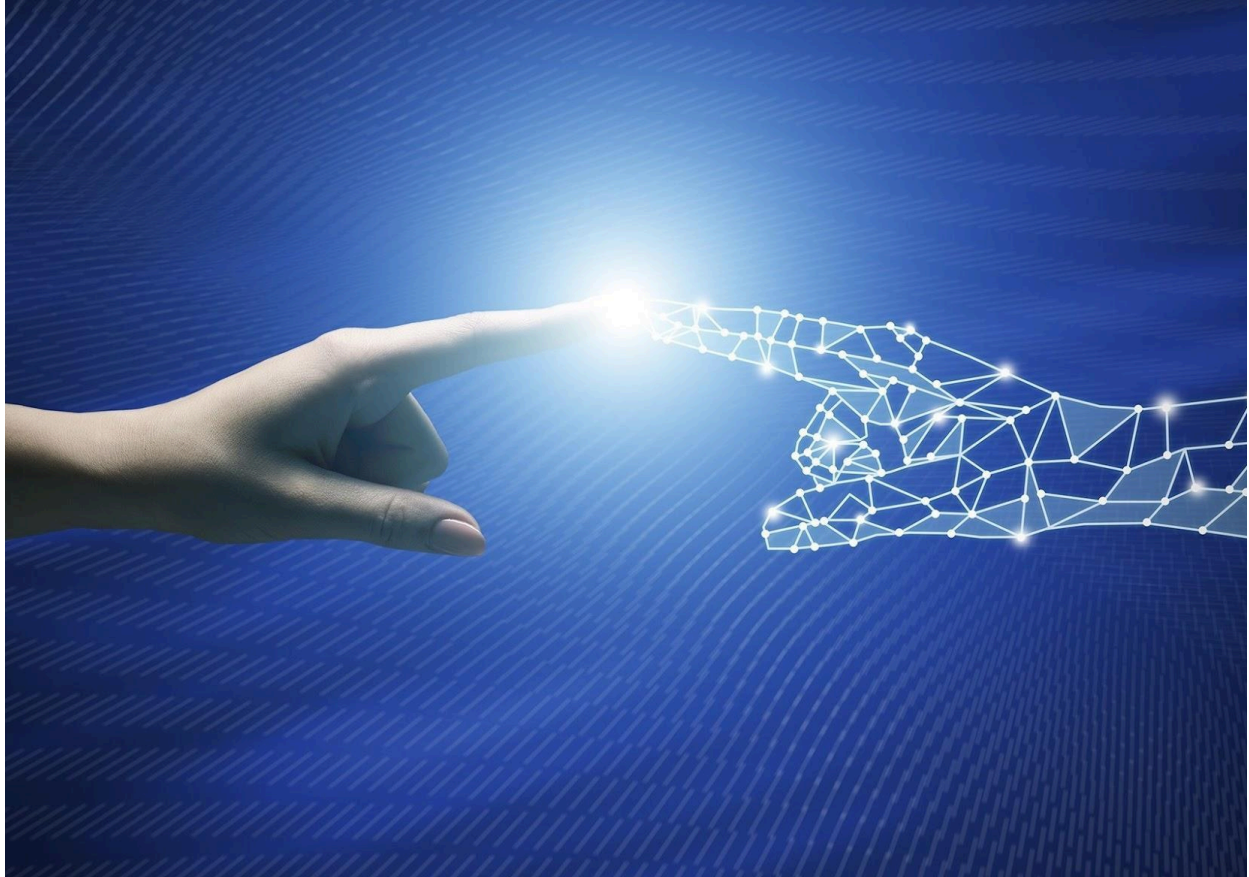


Machine Learning Document

Document containing fundamentals learnt during machine learning



Taledi Mphela

Models:

XgBoost

Components - gradient boosting + decision tree ie. example of ensemble model.

Advantages - not sensitive to multicollinearity, learns non-linear relationships between features well so that you do not have to explicitly feature engineer complex relationships. Works well with parallel processing. Not affected by outliers within the data set. Produces MI scores that form a great basis for feature engineering.

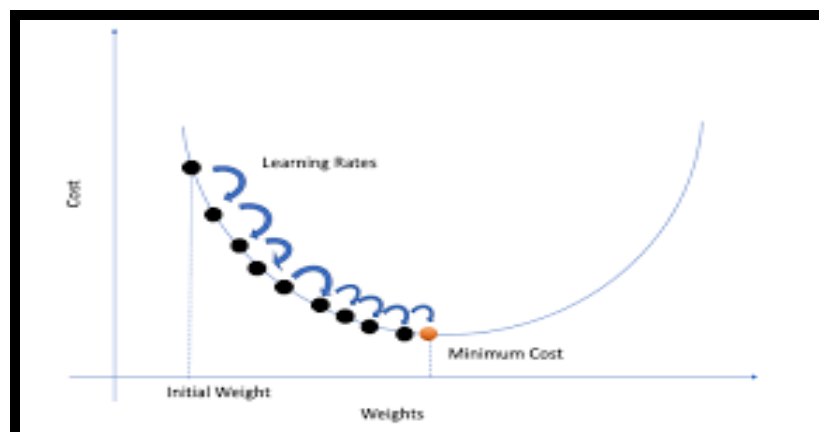
Disadvantages - tends to be highly accurate and may lead you to spurious accuracy. Long training time. Hard to interpret for non-technical members.

Overview: A model that combines gradient boosting and decision trees. Trees are developed based on features and splits occur depending on gain, the sum of the first and second derivative of the loss function. Gradient boosting computes a weak predictor value then gets the error from the prediction and trains subsequent models on residuals then finally makes a prediction by adding up all the errors plus the initial weak prediction.

Illustration: makes a weak prediction with random features -> uses the error of the predictions to train a new small model -> makes a small decision tree model to predict on errors -> continues until error meets criteria.

Hyperparameters:

learning rate - controls that rate at which the model changes losses during stochastic gradient descent. A larger value i.e. 1 corresponds with the model underfitting. A smaller value i.e. 0.01 corresponds with the model overfitting in rare cases.



If you have a larger learning rate, your model is more likely to 'over-step' the minimum loss.

verbose - Number that tells python how many messages to print during training

n_estimators - number of iterations the model performs. Higher values correspond to overfitting, lower values correspond to underfitting.

early_stopping_rounds - a positive integer value that tells the model when it's overfitting i.e. when the validation score stops improving. Goes hand in hand with n_estimators and the eval_set parameter.

eval_set - a comparison data set to score the performance of the model on data unseen. Usually your test training data eg. eval_set = [(Xtest,ytest)]

n_jobs - value either 1 or -1. The former doesn't use all your pc resources, the latter uses all your computer resources to speed up the process

max_depth - the maximum number of nodes developed by a tree, the more nodes, the deeper the thinking and more complex relationships found. Note that larger values correspond to overfitting and are computationally exhaustive. Natural numbers

min_child_weight - numeric value that controls how many complex relationships the model learns in each node. Smaller values correspond to more complexity, larger values correspond to less. Useful in cases of small data sets to prevent overfitting. Natural numbers

colsample_bytree - fraction of features presented in each tree. Introduces randomness and correlation between trees to prevent overfitting. ('[Xgboosting.com](https://xgboost.ai)'). Rational numbers.

reg_alpha - encourages sparse solutions and prevents overfitting. Larger value of alpha, reduce overfitting. Any positive rational value. Uses a penalty that reduces some weights to zero.

reg_lambda - similar to reg_alpha but smoothens out weights.

Scale_pos_weight - motivates your model to search for more of the less frequent class. Formula = $\frac{\text{sum}(0)}{\text{sum}(1)}$

Key metrics for classification:

Look at the formula of metrics, this tells you what we want to optimize.

Precision - $tp/(tp+fp)$ here fp appears in the formula, cases where fp 's are a huge risk, we focus on precision. Adjust by changing the threshold for probabilities. Larger threshold -> fewer false positives -> higher precision

Recall - $tp/(tp+fn)$ here fn appears in the formula, cases where fn 's are a huge risk, we want to focus more on recall

Random Forest-

A decision tree algorithm. First the model creates a sample of N . Selects m features, splits nodes depending on metric then finally makes a prediction on either the mode of each tree for classification or the mean for each tree based on regression. Useful features are ones that ease the splitting at each tree.

Assumption: Random forest requires enough data

Hyperparameters:

`n_estimators` - number of decision trees, the higher the value, the better the predictions but longer training time. Positive integer value

`max_features` - maximum number of features at each node. Positive integer value

`max_depth` - The amount of non-linear interactions you want the tree to discover. Longer training times and risk of overfitting. Positive integer. Default is none and then depends on `min_samples_split`.

`min_samples_split` - Minimum number of samples required for the tree to make a split. Prevents overfitting on small sample sizes. Positive integer value.

`min_samples_leaf` - Controls the number of samples at each leaf. Higher values prevent overfitting. Positive integer value.

MLPClassifier-

Neural network that works best on non-linear relationships between features.

Neural networks operate like our brains. Nodes are seen as where transformations occur. Makes use of forward and back propagation to adjust weights behind predictions. The more layers, the more complex relationships found within data. The drawback is the model tends to overfit.

Advantages - Tends to not be overly accurate. Great practical use. Abundance of online resources and models to use for transfer learning to solve similar problems.

Disadvantages - Does not produce prediction probabilities. Requires a lot of processing power. The model learns from tensors and not actual tabular data which decreases interpretability behind model findings.

Hyperparameters:

hidden_layer_size - specifies number of hidden layers and neurons. More neurons and networks means more complex relationships found within the model.

max_iter - number of epochs

activation function - functions that transform the data from neuron to neuron, common functions include: tanh, relu, sigmoid, softmax

learning rate - how much the model adjust loss, larger values lead to underfitting, lower values lead to overfitting

solver - also known as optimizer, how the model tries to solve loss issue. Common solvers include, Adam, SGD and lbfgs

input_features - the number of features accepted at each node in the learning process. Nodes learn surface level information.

output_features - the required output for the model, for classification and regression usually 1 and for image recognition depends on recognition task and metrics of images.

Credit to [Daniel Bourke](#) for his vast array of free online neural network information.

Light gradient boosting

Components: gradient boosting + decision trees + goss

Ensemble method similar to xgboost. The algorithm uses histogram based learning i.e. turning continuous features into bins -> splits nodes at maximum gain -> samples features from samples with small gradient -> builds it on features.

GOSS(Gradient - based One-Side Sampling) is a learning method that picks features based on their initial gradients of the loss function from the previously 'weak' trained model. The larger the gradient, the more information the model is likely to learn from these features.

Advantages:

Allows for categorical features without any numerical encoding. Works quickly and efficiently even with large datasets. Automatically creates new features.

Disadvantage:

Also leads to spurious accuracy

Hyperparameters:

learning_rate - covered in xgboost model.

boosting - determines boosting algorithm used.

Boosting algorithms include: gbdt - traditional gradient boosting method explained in xgboost model. dart - randomly drops decision trees at different iterations to reduce chances of overfitting. rf - random forest method

num_iterations - similar to n_estimators

objective - binary or multiclass or regression

max_depth - explained above

num_leaves - explained above

metric - the desired metric you would like to use for evaluation.

categorical_features - allows for a categorical feature to be used during training. Useful for approaches where the category has a significant impact on the target variable and you would like the model to emphasize the impact.

AdaBoost

Advantage of AdaBoost is it's not prone to overfitting but a disadvantage is it may underfit the data. Algorithm: random training data -> trains model -> assigns weights depending on accuracy -> repeats

Hyperparameters:

base_estimator - the learning algorithm used to train weak models

n_estimators - number of iterations

learning_rate - explained above

Stacking Classifier:

A sci-kit learn classifier that combines models of your choice into an ensemble method. Automatically ensures no data leakage occurs between training each base model and the meta model.

Hyperparameters:

estimator - A list of the base models you desire to use in the stacking ensemble.

final_estimator - The meta model that learns from predictions from base models.

cv - number of cross validation splits

Advantages:

Improves model performance significantly if models are diverse. Errors are minimized between predictions.

Disadvantages:

If models are not diverse, stacking classifier does not learn build on other models' weaknesses. Computing time.

Useful functions for classification problems:

scikit-learns compute_sample_weight - which returns an output that contains an array of the training data's weights based on the frequency of classes. Lower frequency classes have larger weights, higher frequency classes have smaller weights. Just a procedure to improve your models performance for imbalance data sets.

target encoding - also known as mean encoding provides a group by means of your target variable for each category in the categorical column. **Note that this is a risky encoding as it may lead to data leakage and overfitting. Should never be used in real world applications.**

Out-of-fold:

Crucial for evaluating the models performance on unseen data sets. If the model

performs well on OOF then we can reliably say the model will have a similar performance in production.

The various validation methods below implement OOF for us.

Out-of-fold predictions:

Raw predictions your fold model makes on the validation set. This removes any need for a holdout set for further analysis later. Gives you a real-world outlook on how you perform on the basis the data is representative of the population.

Bagging is a method used with OOF predictions and ensemble methods. This approach is often better for data with high variance. Bagging reduces the impact of noise on different fold models. The ensemble is your different models from each fold. Bagging is just another term for average.

Preventing overfitting/large variance within predictions:

cross validation - this is the process of splitting your data into a train and validation set(hold out data set). Algorithm: model fits on training data -> predicts on validation set -> produces score

Nested cross validation - technique used to determine hyperparameters and model evaluation in different cross validations to prevent leakage. The problem with using grid search cross validation for finding the optimal hyperparameters is the parameters found are associated with data for which the target variable is known. In production, the target is not known.

KFold - Splits the data set in k groups for training and testing.

StratifiedKFold - Automatically balances the target variable within the training and test set of data. Extremely helpful for imbalanced data sets.

GroupKFold - Works well when our data is a collection from different groups, say different doctors, different races etc. Ensures the model does not learn from groups for example a doctor does not appear in the validation and train split.

Coding on a server:

Important terminal commands:

git pull - pulls information from global server to local machine

git add . - collects the new changes made to the information from the server

git commit -m "Comment" - adds comment to new information posted to server

git push - uploads everything to server

Logistic regression solvers: (Stacking models)

lbfgs - memory efficient solver. Fails to converge on large datasets.

liblinear - works well on small datasets with many features. Makes use of L1 regularization.

newton-cg - efficient for multiclass classification and uses L2 regularization.

sag - fast for large data sets

saga - L1 and L2 regularization

L1 regularization - useful for data sets with many features. Prevents overfitting by reducing coefficient estimates to zero if needed.

L2 regularization - forces coefficients to be small but non-zero. Helpful in cases where features may have interaction.